

PythonGetaway

Gateway HTTP multiusuario sobre MetaTrader 5

Referencia completa de la API — 49 endpoints

Version 2.0.0 — multiusuario

Cliente destino: .NET

Todos los ejemplos de este documento son respuestas **reales**
capturadas contra una cuenta MT5 en vivo.

1. Endpoints nuevos: autenticacion y pool (8)

Estos son los endpoints que **no existian** en la version monousuario. Son la puerta de entrada de la API: sin un token de `/auth/login` ninguno de los otros 41 responde.

Resumen del flujo. POST `/auth/login` con las credenciales MT5 del usuario devuelve un `token` . Ese token va en `Authorization: Bearer <token>` en todas las llamadas siguientes. POST `/auth/logout` libera el terminal.

Metodo	Ruta	Necesita token	Para que
POST	<code>/auth/login</code>	no	Autenticarse con credenciales MT5 y reservar un terminal dedicado.
POST	<code>/auth/refresh</code>	si	Reiniciar el reloj de vida maxima. El token no cambia.
POST	<code>/auth/logout</code>	si	Cerrar la sesion y devolver el terminal al pool.
GET	<code>/auth/session</code>	si	Estado de la sesion actual. Renueva la ventana de inactividad.
GET	<code>/auth/sessions</code>	no	Sesiones activas, sin tokens ni credenciales. Vision de operacion.
GET	<code>/</code>	no	Indice del gateway.
GET	<code>/health</code>	no	Sonda: estado del gateway y cuantos usuarios nuevos caben.
GET	<code>/pool</code>	no	Detalle de cada terminal del pool.

La contraseña no se guarda en ningun sitio. Viaja una sola vez del cliente al worker, autentica el terminal y se descarta: no va a disco, ni a los logs, ni queda en memoria del gateway. Si el gateway se reinicia, los usuarios vuelven a autenticarse.

Y por eso mismo: con `--host 0.0.0.0` y sin HTTPS delante, esas credenciales viajan **en claro**. Pon un reverse proxy con TLS.

Autenticacion

POST

/auth/login

sin token

Autenticarse en MT5 con credenciales propias

Reserva un terminal MT5 dedicado para esta cuenta y devuelve un token.

Si la cuenta ya tiene una sesion viva, se devuelve **la misma sesion** en lugar de consumir un segundo slot.

La contraseña no se guarda: viaja una vez hasta el worker, autentica el terminal y se descarta. Nunca aparece en logs.

Cuerpo de la peticion — `application/json`

Campo	Tipo	Obligatorio	Descripcion
login	integer (int64)	si	Numero de cuenta MT5.
password	string	si	Contraseña de la cuenta (de operador o de inversor). No se guarda en ningun sitio: se usa para autenticar el terminal y se descarta.
server	string	si	Nombre exacto del servidor del broker, tal cual aparece en MT5. Distingue mayusculas.

Respuesta — `application/json` (HTTP 200)

Campo	Tipo	Presencia	Descripcion
slot_id	string	siempre	Terminal MT5 asignado a esta sesion.
login	integer (int64)	siempre	Cuenta autenticada.
server	string	siempre	
account	object?	puede faltar	Resumen de la cuenta.
created_at	integer (int64)	siempre	Epoch UTC en segundos del login.
last_seen	integer (int64)	siempre	Epoch UTC de la ultima peticion con este token.
expires_at	integer (int64)	siempre	Epoch UTC en que caduca. Es el MINIMO entre vida maxima e inactividad.
expires_in	integer (int64)	siempre	Segundos que le quedan.
ttl_seconds	integer (int64)	siempre	Vida maxima configurada.
idle_timeout_seconds	integer (int64)	siempre	Ventana de inactividad configurada.
token	string	siempre	Token de sesion. Va en <code>`Authorization: Bearer <token>`</code> en TODAS las llamadas siguientes.
reused	boolean	siempre	<code>`true`</code> si la cuenta ya tenia una sesion viva y se devuelve esa misma en lugar de consumir otro

terminal.

Estructura documentada, no capturada. Un login exitoso exige credenciales reales, así que este ejemplo describe la forma exacta de la respuesta (derivada del código y del esquema), no una ejecución. Lanzando el selftest con `PYGW_TEST_LOGIN` / `PYGW_TEST_PASSWORD` / `PYGW_TEST_SERVER` y regenerando este PDF, se sustituye por una captura real.

Peticion

```
{
  "login": 5054767214,
  "password": "<tu contrasena>",
  "server": "MetaQuotes-Demo"
}
```

Respuesta correcta — HTTP 200

```
{
  "reused": false,
  "token": "kHs9_2FqxV7bN1pQ... (43 caracteres url-safe)",
  "slot_id": "slot-1",
  "login": 5054767214,
  "server": "MetaQuotes-Demo",
  "account": {
    "login": 5054767214,
    "server": "MetaQuotes-Demo",
    "currency": "USD",
    "balance": 96126.17,
    "equity": 96126.17,
    "trade_mode": 0,
    "trade_mode_name": "DEMO",
    "trade_allowed": true
  },
  "created_at": 1789040000,
  "last_seen": 1789040000,
  "expires_at": 1789040900,
  "expires_in": 900,
  "ttl_seconds": 3600,
  "idle_timeout_seconds": 900
}
```

MT5 rechazo las credenciales. `detail.error.mt5\_code` trae el código original (-6 = Authorization failed).

Ejemplo de petición (real)

```
{
  "login": 999999999,
  "password": "*****",
  "server": "MetaQuotes-Demo"
}
```

Ejemplo de ERROR (real, capturado) — HTTP 401

```
{
  "error": {
    "code": "MT5_LOGIN_FAILED",
    "message": "MT5 rechazo las credenciales o no pudo conectar con el servidor.",
    "mt5_code": null,
    "detail": {
      "error": {
        "code": "MT5_ERROR",
        "message": "No se pudo inicializar el terminal MT5: Terminal: Authorization failed",
        "mt5_code": -6,
        "detail": null
      }
    }
  }
}
```

POST /auth/refresh requiere token

Renovar la sesion sin reenviar credenciales

Reinicia el reloj de **vida maxima** de la sesion.

El token **no cambia**: se sigue usando el mismo, con lo que no hay carrera con peticiones en vuelo. Solo se reinicia el contador.

Cuando hace falta:

- La ventana de **inactividad** (15 min) ya se renueva sola con cualquier llamada autenticada. Para eso no hace falta este endpoint. - El tope de **vida maxima** (1 h) NO se renueva solo. Un cliente que este conectado toda la sesion bursatil debe llamar aqui cada cierto tiempo (p.ej. cada 30 min) para no verse obligado a reenviar las credenciales.

Si prefieres que no exista tope absoluto, arranca con `PYGW\_SESSION\_TTL=0`: entonces la sesion solo muere por inactividad y este endpoint es opcional.

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripción
slot_id	string	siempre	Terminal MT5 asignado a esta sesion.
login	integer (int64)	siempre	Cuenta autenticada.
server	string	siempre	
account	object?	puede faltar	Resumen de la cuenta.
created_at	integer (int64)	siempre	Epoch UTC en segundos del login.
last_seen	integer (int64)	siempre	Epoch UTC de la ultima peticion con este token.
expires_at	integer (int64)	siempre	Epoch UTC en que caduca. Es el MINIMO entre vida maxima e inactividad.
expires_in	integer (int64)	siempre	Segundos que le quedan.
ttl_seconds	integer (int64)	siempre	Vida maxima configurada.
idle_timeout_seconds	integer (int64)	siempre	Ventana de inactividad configurada.

Estructura documentada, no capturada. Un login exitoso exige credenciales reales, asi que este ejemplo describe la forma exacta de la respuesta (derivada del codigo y del esquema), no una ejecucion. Lanzando el selftest con `PYGW_TEST_LOGIN` / `PYGW_TEST_PASSWORD` / `PYGW_TEST_SERVER` y regenerando este PDF, se sustituye por una captura real.

Respuesta correcta — HTTP 200

```
{
  "slot_id": "slot-1",
  "login": 5054767214,
  "server": "MetaQuotes-Demo",
  "account": {
    "login": 5054767214,
    "server": "MetaQuotes-Demo",
    "currency": "USD",
    "balance": 96126.17,
    "equity": 96126.17,
    "trade_mode": 0,
    "trade_mode_name": "DEMO",
    "trade_allowed": true
  },
  "created_at": 1789041000,
  "last_seen": 1789041000,
  "expires_at": 1789041900,
  "expires_in": 900,
  "ttl_seconds": 3600,
  "idle_timeout_seconds": 900
}
```

GET /auth/session requiere token

Estado de la sesion actual

Datos de la sesion asociada al token, con lo que le queda de vida. Llamarlo tambien **renueva** la ventana de inactividad.

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripcion
slot_id	string	siempre	Terminal MT5 asignado a esta sesion.
login	integer (int64)	siempre	Cuenta autentificada.
server	string	siempre	
account	object?	puede faltar	Resumen de la cuenta.
created_at	integer (int64)	siempre	Epoch UTC en segundos del login.
last_seen	integer (int64)	siempre	Epoch UTC de la ultima peticion con este token.
expires_at	integer (int64)	siempre	Epoch UTC en que caduca. Es el MINIMO entre vida maxima e inactividad.
expires_in	integer (int64)	siempre	Segundos que le quedan.
ttl_seconds	integer (int64)	siempre	Vida maxima configurada.
idle_timeout_seconds	integer (int64)	siempre	Ventana de inactividad configurada.

Estructura documentada, no capturada. Un login exitoso exige credenciales reales, asi que este ejemplo describe la forma exacta de la respuesta (derivada del codigo y del esquema), no una ejecucion. Lanzando el selftest con `PYGW_TEST_LOGIN` / `PYGW_TEST_PASSWORD` / `PYGW_TEST_SERVER` y regenerando este PDF, se sustituye por una captura real.

Respuesta correcta — HTTP 200

```
{
  "slot_id": "slot-1",
  "login": 5054767214,
  "server": "MetaQuotes-Demo",
  "account": {
    "login": 5054767214,
    "server": "MetaQuotes-Demo",
    "currency": "USD",
    "balance": 96126.17,
    "equity": 96126.17,
    "trade_mode": 0,
    "trade_mode_name": "DEMO",
    "trade_allowed": true
  },
  "created_at": 1789040000,
  "last_seen": 1789040420,
  "expires_at": 1789041320,
  "expires_in": 900,
  "ttl_seconds": 3600,
  "idle_timeout_seconds": 900
}
```

POST /auth/logout requiere token

Cerrar sesion y liberar el terminal

Cierra la sesion MT5 y devuelve el slot al pool.

Por defecto el terminal se reinicia para que no quede ninguna cuenta conectada (`PYGW_RECYCLE_ON_LOGOUT``), asi que el slot tarda unos segundos en volver a estar disponible.

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripcion
logged_out	boolean	siempre	Siempre `true` si el token era valido.
recycled	boolean	siempre	Si el terminal se reinicia para no dejar ninguna cuenta conectada (<code>PYGW_RECYCLE_ON_LOGOUT`</code>).

Estructura documentada, no capturada. Un login exitoso exige credenciales reales, asi que este ejemplo describe la forma exacta de la respuesta (derivada del codigo y del esquema), no una ejecucion. Lanzando el selftest con `PYGW_TEST_LOGIN` / `PYGW_TEST_PASSWORD` / `PYGW_TEST_SERVER` y regenerando este PDF, se sustituye por una captura real.

Respuesta correcta — HTTP 200

```
{
  "logged_out": true,
  "recycled": true
}
```

GET /auth/sessions sin token

Sesiones activas (sin datos sensibles)

Vision de operacion: cuantas sesiones hay y en que slot esta cada una. No expone tokens ni credenciales.

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripcion
count	integer (int64)	siempre	Numero de sesiones activas.
sessions	array<object>	siempre	Una entrada por sesion. No expone tokens ni credenciales.
sessions[].slot_id	string		
sessions[].login	integer (int64)		
sessions[].server	string		
sessions[].created_at	integer (int64)		
sessions[].last_seen	integer (int64)		
sessions[].expires_at	integer (int64)		
sessions[].expires_in	integer (int64)		
sessions[].ttl_seconds	integer (int64)		
sessions[].idle_timeout_seconds	integer (int64)		

Estructura documentada, no capturada. Un login exitoso exige credenciales reales, así que este ejemplo describe la forma exacta de la respuesta (derivada del código y del esquema), no una ejecución. Lanzando el selftest con `PYGW_TEST_LOGIN` / `PYGW_TEST_PASSWORD` / `PYGW_TEST_SERVER` y regenerando este PDF, se sustituye por una captura real.

Respuesta correcta — HTTP 200

```
{
  "count": 1,
  "sessions": [
    {
      "slot_id": "slot-1",
      "login": 5054767214,
      "server": "MetaQuotes-Demo",
      "created_at": 1789040000,
      "last_seen": 1789040420,
      "expires_at": 1789041320,
      "expires_in": 900,
      "ttl_seconds": 3600,
      "idle_timeout_seconds": 900
    }
  ]
}
```

Gateway y pool

GET / sin token

Indice del gateway

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripcion
name	string	siempre	
version	string	siempre	
mode	string	siempre	
docs	string	siempre	Ruta de la documentacion interactiva.
openapi	string	siempre	Ruta del esquema OpenAPI.
health	string	siempre	
login	string	siempre	Por donde empezar.
pool	object	siempre	Recuento de slots por estado.

Indice. No requiere token.

Ejemplo de respuesta (real) — HTTP 200

```
{
  "name": "PythonGetaway - MT5 Gateway",
  "version": "2.0.0",
  "mode": "multiusuario (pool de terminales)",
  "docs": "/docs",
  "openapi": "/openapi.json",
  "health": "/health",
  "login": "POST /auth/login",
  "pool": {
    "IDLE": 2
  }
}
```

GET /health sin token

Estado del gateway y del pool

No requiere token: es el endpoint de sonda.
`available` es cuantos usuarios nuevos pueden autenticarse ahora mismo.

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripcion
status	string	siempre	`ok` si queda algun slot libre, `saturado` si no.
pool	object	siempre	
pool.size	integer (int64)		Numero total de slots (usuarios concurrentes maximos).
pool.states	object		Recuento de slots por estado.
pool.available	integer (int64)		Slots en IDLE: cuantos usuarios NUEVOS pueden autenticarse ahora mismo.
pool.slots	array<object>		Detalle de cada slot.
pool.slots[].slot_id	string		Identificador del slot.
pool.slots[].state	string		COLD WARMING IDLE BUSY RECYCLING FAILED. Solo los IDLE aceptan un login nuevo.
pool.slots[].port	integer (int64)		Puerto local del worker. Uso interno; no se expone fuera.
pool.slots[].login	integer (int64)?		Cuenta autenticada en este slot, o null si esta libre.
pool.slots[].server	string?		Servidor del broker, o null si esta libre.
pool.slots[].seconds_in_state	number (double)		Segundos que lleva en el estado actual.
pool.slots[].error	string?		Motivo del fallo si el estado es FAILED.
sessions_active	integer (int64)	siempre	Sesiones de usuario vivas.
allow_real_account	boolean	siempre	Si el trading esta permitido en cuentas REALES.
recycle_on_logout	boolean	siempre	Si el terminal se reinicia al cerrar sesion.
session_ttl_seconds	integer	siempre	Vida maxima de un token.

	(int64)		
session_idle_timeout_seconds	integer (int64)	siempre	Caducidad por inactividad.

Sonda del gateway. No requiere token. `available` es cuantos usuarios nuevos pueden autenticarse ahora mismo.

Ejemplo de respuesta (real) — HTTP 200

```
{
  "status": "ok",
  "pool": {
    "size": 2,
    "states": {
      "IDLE": 2
    },
    "available": 2,
    "slots": [
      {
        "slot_id": "slot-1",
        "state": "IDLE",
        "port": 8101,
        "login": null,
        "server": null,
        "seconds_in_state": 30.8,
        "error": null
      },
      {
        "slot_id": "slot-2",
        "state": "IDLE",
        "port": 8102,
        "login": null,
        "server": null,
        "seconds_in_state": 30.8,
        "error": null
      }
    ]
  },
  "sessions_active": 0,
  "allow_real_account": false,
  "recycle_on_logout": true,
  "session_ttl_seconds": 3600,
  "session_idle_timeout_seconds": 900
}
```

GET /pool sin token

Detalle del pool de terminales

Estado de cada slot. Util para operar y dimensionar.

Respuesta — application/json (HTTP 200)

Campo	Tipo	Presencia	Descripcion
size	integer (int64)	siempre	Numero total de slots (usuarios concurrentes maximos).
states	object	siempre	Recuento de slots por estado.
available	integer (int64)	siempre	Slots en IDLE: cuantos usuarios NUEVOS pueden autenticarse ahora mismo.
slots	array<object>	siempre	Detalle de cada slot.
slots[].slot_id	string		Identificador del slot.
slots[].state	string		COLD WARMING IDLE BUSY RECYCLING FAILED. Solo los IDLE aceptan un login nuevo.
slots[].port	integer (int64)		Puerto local del worker. Uso interno; no se expone fuera.
slots[].login	integer (int64)?		Cuenta autenticada en este slot, o null si esta libre.
slots[].server	string?		Servidor del broker, o null si esta libre.
slots[].seconds_in_state	number (double)		Segundos que lleva en el estado actual.
slots[].error	string?		Motivo del fallo si el estado es FAILED.

Estado de cada terminal del pool. Util para operar y dimensionar.

Ejemplo de respuesta (real) — HTTP 200

```
{
  "size": 2,
  "states": {
    "IDLE": 2
  },
  "available": 2,
  "slots": [
    {
      "slot_id": "slot-1",
      "state": "IDLE",
      "port": 8101,
      "login": null,
      "server": null,
      "seconds_in_state": 30.8,
      "error": null
    },
    {
      "slot_id": "slot-2",
      "state": "IDLE",
      "port": 8102,
      "login": null,
      "server": null,
      "seconds_in_state": 30.8,
      "error": null
    }
  ]
}
```

2. Arrancar y parar

Dos comandos. El primero levanta el gateway, que a su vez provisiona y arranca **el pool completo**: un terminal MT5 y un proceso worker por cada usuario concurrente.

Arrancar todo

```
cd /home/borislav/Desktop/TradingAI/PythonGetaway
./scripts/start.sh
```

No devuelve el control hasta que hay al menos un slot disponible. El primer arranque tarda mas: clona la instalacion de MT5 una vez por slot.

Opciones

Opcion	Efecto
--pool 5	Usuarios concurrentes (por defecto 3). Cada uno cuesta ~400 MB de RAM.
--port 9000	Puerto publico (por defecto 8000).
--host 0.0.0.0	Escuchar en toda la red. Por defecto solo 127.0.0.1 .

--allow-real	Permitir operar en cuentas REALES. Por defecto se bloquea con 403.
--foreground	No daemonizar: logs en la terminal, Ctrl+C para parar.

Parar todo

```
cd /home/borislav/Desktop/TradingAI/PythonGetaway
./scripts/stop.sh
```

Para el gateway (que cierra su pool de forma ordenada) y despues barre cualquier terminal o worker que hubiera quedado suelto. Las instancias MT5 clonadas se conservan para el proximo arranque.

Comprobar que esta vivo

```
curl http://127.0.0.1:8000/health # estado del gateway y del pool
curl http://127.0.0.1:8000/pool # detalle de cada slot
```

Probarlo de punta a punta

```
# solo lo que no necesita cuenta (gateway, pool, rutas de error)
python3 scripts/selftest.py

# completo: hace login real y prueba los 41 endpoints de cuenta
PYGW_TEST_LOGIN=5054767214 PYGW_TEST_PASSWORD='...' PYGW_TEST_SERVER=MetaQuotes-Demo \
python3 scripts/selftest.py

# completo pero sin enviar ninguna orden
... python3 scripts/selftest.py --no-trading
```

Aviso: sin --no-trading el selftest abre y cierra posiciones reales en la cuenta indicada. Se niega a operar si la cuenta no es DEMO. La contrasena se lee del entorno y nunca se imprime ni se guarda en los ejemplos.

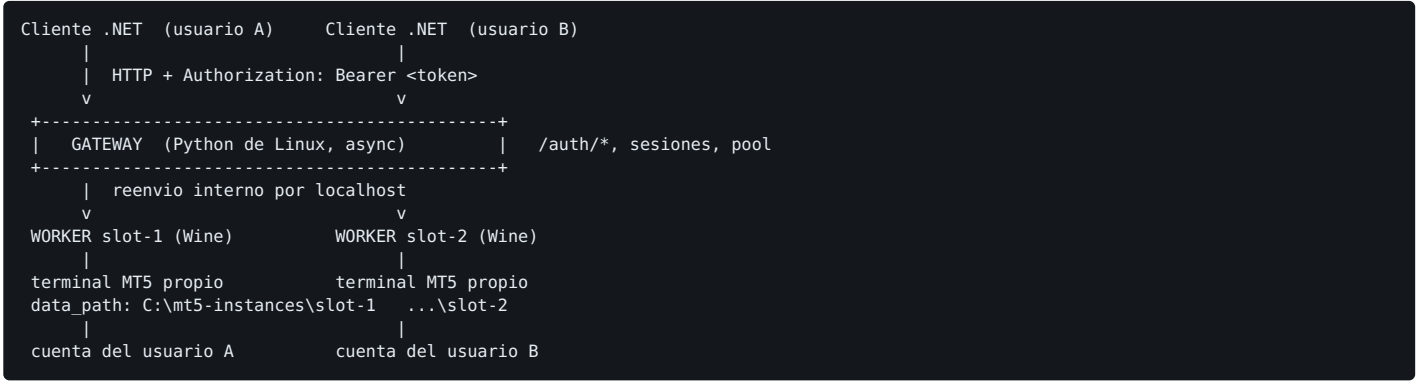
Recursos en marcha

Recurso	URL
API	http://127.0.0.1:8000
Swagger UI (interactivo)	http://127.0.0.1:8000/docs
Esquema OpenAPI	http://127.0.0.1:8000/openapi.json
Log del gateway	logs/gateway.log
Log de cada slot	logs/slot-N-worker.log , logs/slot-N-terminal.log

3. Como esta montado (y por que)

El problema que resuelve esta arquitectura. La libreria MetaTrader5 de Python es un **singleton de proceso**: mt5.login() cambia la cuenta de *todo el proceso y de su terminal*. Si varios usuarios compartieran proceso, el login de uno cambiaria la sesion de otro, y un «cerrar posicion» podría ejecutarse **contra la cuenta equivocada**. Verificado: dos procesos atados al mismo terminal comparten cuenta.

Por eso cada sesion recibe **su propio terminal MT5 y su propio proceso**. Es la unica forma de servir N cuentas a la vez.



El **gateway** corre en el Python de Linux: solo enruta, no necesita la libreria MT5 y gana concurrencia real. Los **workers** corren bajo Wine, que es donde vive la DLL del terminal.

Coste por usuario concurrente

Recurso	Coste	Comentario
RAM	~400 MB	~265 MB el terminal + ~135 MB el worker
Disco	~300 MB	Clon de la instalacion, mas la cache de historico que descargue

Arranque	10-20 s	Ocurre al calentar el pool, no dentro del login
Login	1-3 s	El terminal ya esta caliente: solo se autentica

Dimensiona `--pool` con la RAM disponible. Para mas usuarios de los que quepan en una maquina, replica el gateway completo detras de un balanceador: cada instancia gestiona su propio pool y las sesiones son independientes.

Estados de un slot

Estado	Significado
COLD	Sin procesos.
WARMING	Terminal y worker arrancando.
IDLE	Listo y sin cuenta: disponible para un login.
BUSY	Autenticado con una cuenta concreta.
RECYCLING	Cerrando sesion y reiniciando el terminal.
FAILED	No se pudo levantar. El motivo va en <code>error</code> .

Variables de entorno

Variable	Por defecto	Para que sirve
PYGW_HOST / PYGW_PORT	127.0.0.1 / 8000	Interfaz y puerto publicos.
PYGW_POOL_SIZE	3	Usuarios concurrentes.
PYGW_SESSION_TTL	3600	Vida maxima de un token, en segundos.
PYGW_SESSION_IDLE	900	Caducidad por inactividad. Libera el slot solo.
PYGW_RECYCLE_ON_LOGOUT	1	Reiniciar el terminal al cerrar sesion para que no quede ninguna cuenta conectada.
PYGW_ALLOW_REAL	0	Si es <code>0</code> y la cuenta es REAL, el trading devuelve 403 . La lectura sigue funcionando.
PYGW_MAX_LOGIN_ATTEMPTS	5	Intentos fallidos por cuenta antes de devolver 429.
PYGW_MAGIC	20260910	Magic number por defecto de las ordenes.
PYGW_DEVIATION	20	Slippage maximo por defecto, en puntos.

4. Autenticacion en detalle

Cada usuario aporta **sus propias credenciales MT5**. El flujo tiene tres pasos:

```
1. POST /auth/login {login, password, server} -> { "token": "...", ... }
2. Cualquier endpoint con Authorization: Bearer <token>
3. POST /auth/logout (o dejar que caduque) -> libera el terminal
```

4.1 Que pasa con la contraseña

No se guarda en ningun sitio. Viaja una sola vez del cliente al worker, se usa para autenticar el terminal y se descarta. No se escribe en disco, ni en los logs, ni queda en memoria del gateway. Lo unico que persiste mientras dura la sesion es el token y a que slot apunta.

Consecuencia asumida: si el gateway se reinicia, todos los usuarios vuelven a autenticarse. Es el precio de no ser un almacen de credenciales.

TLS es obligatorio si sales de localhost. Con `--host 0.0.0.0` y sin HTTPS delante, las credenciales MT5 de tus usuarios viajan **en claro**. Pon un reverse proxy (nginx, Caddy, Traefik) con certificado. El gateway no termina TLS por si mismo, a proposito: esa responsabilidad es del proxy.

4.2 Ciclo de vida del token

Concepto	Valor por defecto	Detalle
Vida maxima	3600 s	Desde el login, pase lo que pase.
Inactividad	900 s	Ventana deslizante : cada peticion la renueva.
Caducidad efectiva	—	La que llegue antes de las dos. Va en <code>expires_at</code> / <code>expires_in</code> .
Al caducar	—	Un barrido cierra la sesion y libera el slot . Sin esto, un cliente que se desconecta sin logout agotaria el pool.

Un login con una cuenta que **ya tiene sesion viva** devuelve la misma sesion (`reused: true`) en lugar de consumir un segundo slot.

4.3 Errores especificos de autenticacion

HTTP	code	Que hacer
401	MISSING_TOKEN	Falta la cabecera <code>Authorization</code> .
401	INVALID_OR_EXPIRED_TOKEN	Volver a <code>/auth/login</code> .
401	MT5_LOGIN_FAILED	Credenciales o servidor incorrectos. <code>detail.error.mt5_code</code> trae el codigo de MT5 (<code>-6</code> = Authorization failed).

429	T00_MANY_LOGIN_ATTEMPTS	5 fallos en 5 minutos para esa cuenta. Esperar.
503	P00L_EXHAUSTED	No hay terminales libres. Cerrar sesiones o subir --pool . detail trae el estado del pool.
409	SESSION_SLOT_LOST	El terminal de esa sesion se reinicio. Volver a autenticarse.
403	REAL_ACCOUNT_BLOCKED	Cuenta REAL sin PYGW_ALLOW_REAL=1 . La lectura sigue funcionando.

4.4 Cabeceras que devuelve el gateway

Toda respuesta proxificada lleva X-Slot-Id y X-Account-Login : sirven para depurar y para confirmar contra que cuenta se ejecuto la llamada.

4.5 Ejemplo completo en C#

```
var http = new HttpClient { BaseAddress = new Uri("https://tu-host/") };

// 1. login
var login = await http.PostAsJsonAsync("auth/login", new {
    login = 5054767214L, password = pass, server = "MetaQuotes-Demo" });
var session = await login.Content.ReadFromJsonAsync<SessionDto>();

// 2. el token va en TODAS las llamadas siguientes
http.DefaultRequestHeaders.Authorization =
    new AuthenticationHeaderValue("Bearer", session.Token);

var account = await http.GetFromJsonAsync<JsonElement>("account");
var open = await http.PostAsJsonAsync("trading/open", new {
    symbol = "EURUSD", side = "BUY", volume = 0.10, sl_points = 300 });

// 3. comprobar SIEMPRE success, no solo el codigo HTTP
var result = await open.Content.ReadFromJsonAsync<JsonElement>();
if (!result.GetProperty("success").GetBoolean())
    throw new Exception(result.GetProperty("retcode_name").GetString());

// 4. liberar el terminal al terminar
await http.PostAsync("auth/logout", null);
```

5. Convenciones de datos

5.1 Fechas y horas

MT5 devuelve los tiempos como **epoch en segundos**, pero medidos con el reloj del **servidor del broker**, no en UTC. El gateway **no inventa** una conversion de zona horaria. Cada campo temporal se expone dos veces:

Campo	Tipo	Ejemplo	Significado
time	integer (int64)	1789037400	Epoch crudo, tal cual lo da MT5.
time_iso	string	"2026-09-10T14:30:00"	El mismo instante formateado, sin sufijo Z .
time_msc	integer (int64)	1789037400123	Epoch en milisegundos (ticks y deals).

En .NET: deserializa time_iso como DateTime con DateTimeKind.Unspecified . La ausencia deliberada de Z senala que es hora de servidor. Si necesitas UTC real, aplica el offset de tu broker. Para los parametros *_epoch de entrada, envia siempre el epoch en segundos.

5.2 Formato de error (identico en gateway y workers)

```
{
  "error": {
    "code": "NOT_FOUND",
    "message": "No hay ninguna posicion abierta con ticket 999999999.",
    "mt5_code": null,
    "detail": null
  }
}
```

HTTP	Code	Cuando
400	BAD_REQUEST	Parametro invalido (volumen fuera de rango, timeframe inexistente, rango de fechas al revés).
404	NOT_FOUND	Ticket o simbolo inexistente.
422	(FastAPI)	El cuerpo JSON no cumple el esquema. Formato propio de FastAPI.
500	INTERNAL_ERROR	Fallo no previsto. Queda en los logs.
502	MT5_ERROR / WORKER_UNREACHABLE	MT5 devolvio error, o el worker no responde.
503	NOT_CONNECTED	El terminal perdio la conexion con el broker.
504	WORKER_TIMEOUT	El worker no respondio a tiempo.

5.3 Resultado de las ordenes — el punto que mas confunde

Un HTTP 200 no significa que la orden se haya ejecutado. Significa que MT5 recibio la peticion y respondio. Comprueba **siempre** el campo `success` (o `retcode`).

Campo	Tipo	Significado
<code>success</code>	boolean	<code>true</code> solo con <code>retcode</code> 10008, 10009 o 10010.
<code>retcode</code>	integer	Codigo devuelto por el servidor de trading.
<code>retcode_name</code>	string	El mismo codigo en texto: <code>DONE</code> , <code>NO_MONEY</code> , <code>INVALID_STOPS</code> ...
<code>order</code>	integer (int64)	Ticket de la orden generada.
<code>deal</code>	integer (int64)	Ticket del deal (la ejecucion real).
<code>position</code>	integer (int64)	Ticket de la posicion resultante. Es el que usaras para cerrar o mover SL/TP.
<code>volume</code>	number	Volumen realmente ejecutado.
<code>price</code>	number	Precio real de ejecucion.
<code>request</code>	object	Lo que el gateway envio de verdad a MT5. Util para depurar.

`/trading/check` es la excepcion: al no enviar nada, su exito es `retcode == 0` (`retcode_name`: "OK").

5.4 Volúmenes y precios

- El **volumen** se ajusta al `volume_step` del simbolo **siempre hacia abajo**. Redondear hacia arriba aumentaria el riesgo sin que el cliente lo pida. Si el resultado queda por debajo de `volume_min`, la peticion se rechaza con **400**.
- SL y TP** se envian como **precio absoluto**. En `/trading/open` tambien se aceptan como distancia en puntos (`sl_points`, `tp_points`); si mandas ambos, gana el precio absoluto.
- Un **punto** es `symbol_info.point`. Para EURUSD de 5 digitos, 1 punto = 0,00001, luego **1 pip = 10 puntos**.
- Para **quitar** un SL o un TP, envia `0`. Para **dejarlo como esta**, envia `null` u omite el campo.

5.5 Generar el cliente .NET automaticamente

El gateway publica un esquema OpenAPI 3.1 completo (auth + los 41 endpoints de cuenta), con el esquema de seguridad `bearerAuth` ya declarado:

```
nswag openapi2csclient /input:http://127.0.0.1:8000/openapi.json \
                        /classname:Mt5Client /namespace:PythonGetaway /output:Mt5Client.cs

# o Refitter
refitter --namespace PythonGetaway http://127.0.0.1:8000/openapi.json
```

Las respuestas de lectura se devuelven como objetos JSON abiertos: los campos que publica MT5 varian entre brokers y builds del terminal, y cerrar el esquema haria que el gateway **perdiera** campos que el broker si esta enviando. Si prefieres tipos estrictos en .NET, modela solo los campos que uses y deja el resto en un `Dictionary<string, JsonElement>`.

6. Referencia del resto de endpoints (41)

Grupo	Endpoints
Cuenta, terminal y estado	5
Simbolos	4
Datos de mercado	8
Posiciones abiertas	3
Ordenes pendientes	3
Historico	5
Trading manual	13

6.1 Cuenta, terminal y estado

GET **/account** requiere token

Datos completos de la cuenta

Balance, equity, margen, apalancamiento, divisa, tipo de cuenta...

`trade\_mode` 0=DEMO 1=CONTEST 2=REAL (tambien como `trade\_mode\_name`).

Sin cabecera Authorization.

Ejemplo de ERROR (real, capturado) — HTTP 401

```
{
  "error": {
    "code": "MISSING_TOKEN",
    "message": "Falta la cabecera 'Authorization: Bearer <token>'. Obten un token en POST /auth/login.",
    "mt5_code": null,
    "detail": null
  }
}
```

GET **/config** requiere token

Configuracion efectiva del gateway

Lo que el gateway tiene cargado. Nunca expone la contraseña.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET **/last-error** requiere token

Ultimo error de la libreria MT5

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET **/terminal** requiere token

Datos del terminal MT5

Build, rutas, permisos (trade_allowed, dlls_allowed), ping, trafico...

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET **/version** requiere token

Version de la API de MetaTrader5

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

6.2 Simbolos

GET /symbols requiere token

Lista de simbolos del broker

Con `names\_only=false` y sin `group` esto devuelve MUCHOS megas -- el broker de prueba expone >12.000 simbolos. Filtra siempre que puedas.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
group	query	string?	no	—	Filtro estilo MT5, admite comodines. Ej: `*USD*`, `*,!*EUR*` (todos menos los que lleven EUR).
only_visible	query	boolean	no	false	true = solo los que estan en Market Watch.
names_only	query	boolean	no	true	true = solo nombres (rapido). false = especificacion completa de cada simbolo (pesado: son miles).

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET /symbols/total requiere token

Numero total de simbolos

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET /symbols/{symbol} requiere token

Especificacion completa de un simbolo

Digitos, punto, spread, volumen min/max/step, margen, swaps, sesion, modo de llenado permitido, divisas base/beneficio... todo lo que MT5 sabe.

Selecciona el simbolo en Market Watch si aun no lo estaba.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /symbols/{symbol}/select requiere token

Anadir o quitar un simbolo del Market Watch

Un simbolo que no esta en Market Watch no devuelve cotizaciones ni velas. Los endpoints de lectura ya lo seleccionan solos; esto es control manual.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	
enable	query	boolean	no	true	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

6.3 Datos de mercado

GET `/market/book/{symbol}` requiere token

Profundidad de mercado (DOM / Level II)

Suscribe el simbolo al libro, lee y devuelve los niveles.

Muchos brokers de Forex retail NO publican profundidad: en ese caso ``entries`` viene vacio y ``subscribed`` indica si la suscripcion se acepto. No es un error del gateway.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/market/candles/{symbol}` requiere token

Velas OHLCV por posicion (las N mas recientes)

La vela de indice 0 es la que se esta formando AHORA y cambia a cada tick.

Si vas a tomar decisiones sobre velas cerradas, usa ``include_current=false`` (equivale a pedir desde ``start_pos=1``).

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	
timeframe	query	string	no	"M15"	Timeframe: M1, M2, M3, M4, M5, M6, M10, M12, M15, M20, M30, H1, H2, H3, H4, H6, H8, H12, D1, W1, MN1
count	query	integer (int64)	no	500	Numero de velas.
start_pos	query	integer (int64)	no	0	0 = la vela EN CURSO (aun sin cerrar). 1 = la ultima vela cerrada.
include_current	query	boolean	no	true	false = descarta la vela en formacion. Util para no operar con una vela incompleta.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/market/candles/{symbol}/from` requiere token

Velas OHLCV desde una fecha

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	
timeframe	query	string	no	"M15"	Timeframe: M1, M2, M3, M4, M5, M6, M10, M12, M15, M20, M30, H1, H2, H3, H4, H6, H8, H12, D1, W1, MN1
from_epoch	query	integer (int64)	si	—	Epoch en segundos (hora de servidor).
count	query	integer (int64)	no	500	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/market/candles/{symbol}/range` requiere token

Velas OHLCV entre dos fechas

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	
timeframe	query	string	no	"M15"	Timeframe: M1, M2, M3, M4, M5, M6, M10, M12, M15, M20, M30, H1, H2, H3, H4, H6, H8, H12, D1, W1, MN1
from_epoch	query	integer (int64)	si	—	Epoch en segundos (hora de servidor).
to_epoch	query	integer (int64)	si	—	Epoch en segundos (hora de servidor).

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/market/tick/{symbol}` requiere token

Ultimo tick (cotizacion actual)

bid, ask, last, volume, spread calculado y hora del tick.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/market/ticks` requiere token

Ultimo tick de varios simbolos a la vez

Una sola llamada en lugar de N. Los simbolos que fallen se reportan en `errors` en vez de tumbar la peticion entera.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbols	query	string	si	—	Nombres separados por coma. Ej: `EURUSD,GBPUSD,USDJPY`

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/market/ticks/{symbol}/from` requiere token

Ticks crudos desde una fecha

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	
from_epoch	query	integer (int64)	si	—	Epoch en segundos (hora de servidor).
count	query	integer (int64)	no	1000	
flags	query	string	no	"ALL"	ALL INFO TRADE. ALL devuelve todos los ticks.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET /market/ticks/{symbol}/range requiere token

Ticks crudos entre dos fechas

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	path	string	si	—	
from_epoch	query	integer (int64)	si	—	
to_epoch	query	integer (int64)	si	—	
flags	query	string	no	"ALL"	ALL INFO TRADE

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

6.4 Posiciones abiertas

GET /positions requiere token

Posiciones abiertas

`symbol` y `group` son mutuamente excluyentes: si mandas los dos, manda `symbol`. `profit` ya viene neto de swap? No: `profit` es el flotante SIN swap ni comision -- esos van en sus propios campos.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	query	string?	no	—	Filtrar por simbolo exacto.
group	query	string?	no	—	Filtro con comodines. Ej: `*USD*`
magic	query	integer (int64)?	no	—	Filtrar por magic number (se aplica en el gateway).

Token inexistente o caducado.

Ejemplo de ERROR (real, capturado) — HTTP 401

```
{
  "error": {
    "code": "INVALID_OR_EXPIRED_TOKEN",
    "message": "El token no existe o ha caducado. Vuelve a autenticarte.",
    "mt5_code": null,
    "detail": null
  }
}
```

GET /positions/total requiere token

Numero de posiciones abiertas

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET /positions/{ticket} requiere token

Una posicion por ticket

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

6.5 Ordenes pendientes

GET `/orders` requiere token

Ordenes pendientes activas

Solo ordenes VIVAS (limit/stop aun no activadas). Las ya ejecutadas o canceladas estan en `/history/orders``.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
symbol	query	string?	no	—	
group	query	string?	no	—	Filtro con comodines. Ej: <code>`*USD*`</code>
magic	query	integer (int64)?	no	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/orders/total` requiere token

Numero de ordenes pendientes

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/orders/{ticket}` requiere token

Una orden pendiente por ticket

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

6.6 Historico

GET `/history/deals` requiere token

Deals historicos (las ejecuciones reales)

Un deal es un hecho consumado (entrada o salida). El P&L realizado de una posicion es la suma de ``profit + swap + commission + fee`` de sus deals.

Los movimientos de saldo (deposito, retirada) tambien aparecen aqui, con ``type_name = "BALANCE"``.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
from_epoch	query	integer (int64)?	no	—	Epoch s. Por defecto: hace 30 dias.
to_epoch	query	integer (int64)?	no	—	Epoch s. Por defecto: ahora + 1 dia.
symbol	query	string?	no	—	
group	query	string?	no	—	
ticket	query	integer (int64)?	no	—	
position	query	integer (int64)?	no	—	Todos los deals de una posicion: sirve para reconstruir su P&L real.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/history/deals/total` requiere token

Numero de deals en un rango

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
from_epoch	query	integer (int64)?	no	—	
to_epoch	query	integer (int64)?	no	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/history/orders` requiere token

Ordenes historicas (ejecutadas, canceladas, expiradas)

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
from_epoch	query	integer (int64)?	no	—	Epoch s. Por defecto: hace 30 dias.
to_epoch	query	integer (int64)?	no	—	Epoch s. Por defecto: ahora + 1 dia.
symbol	query	string?	no	—	
group	query	string?	no	—	Filtro con comodines, ej. `*USD*`
ticket	query	integer (int64)?	no	—	Una orden concreta (ignora el rango de fechas).
position	query	integer (int64)?	no	—	Todas las ordenes de una posicion (ignora el rango).

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/history/orders/total` requiere token

Numero de ordenes historicas en un rango

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
from_epoch	query	integer (int64)?	no	—	
to_epoch	query	integer (int64)?	no	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

GET `/history/position/{position_id}` requiere token

Vida completa de una posicion cerrada

Reune las ordenes y los deals de una posicion y calcula su P&L realizado.

Es la forma comoda de auditar "que paso exactamente con el ticket X".

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
position_id	path	integer (int64)	si	—	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

6.7 Trading manual

POST /trading/calc-margin requiere token

Margen requerido por una operacion

Cuanto margen bloquearia la operacion. No envia nada al mercado.

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
symbol	string	si	
side	"BUY" "SELL"	si	
volume	number (double)	si	
price	number (double)?	no	Precio de referencia. null = precio de mercado actual.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/calc-profit requiere token

Beneficio teorico de una operacion

P&L que daria ir de `price\_open` a `price\_close`. No envia nada.

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
symbol	string	si	
side	"BUY" "SELL"	si	
volume	number (double)	si	
price_open	number (double)	si	
price_close	number (double)	si	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/check requiere token

Validar una orden SIN enviarla

Simula la orden contra el servidor: dice si pasaria, cuanto margen consumiria y como quedaria el balance. ****No envia nada al mercado.****

Es la forma segura de validar desde .NET antes de ejecutar de verdad.

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
symbol	string	si	Nombre exacto del simbolo en el broker.
side	"BUY" "SELL"	si	BUY o SELL. Ejecucion a mercado.
volume	number (double)	si	Lotes. Se ajusta al volume_step del simbolo, siempre hacia abajo.
sl	number (double)?	no	Stop Loss como PRECIO absoluto. null = sin SL.
tp	number (double)?	no	Take Profit como PRECIO absoluto. null = sin TP.
sl_points	integer (int64)?	no	Alternativa a `sl`: distancia en puntos desde el precio de entrada. Se ignora si `sl` viene informado.
tp_points	integer (int64)?	no	Alternativa a `tp`: distancia en puntos desde el precio de entrada.
deviation	integer (int64)?	no	Slippage maximo aceptado, en puntos. Por defecto el del gateway (20).
magic	integer (int64)?	no	Magic number de la orden. Por defecto el del gateway.
comment	string?	no	Comentario libre. MT5 lo trunca a 31 caracteres.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST `/trading/close-all` requiere token

Cerrar todas las posiciones (opcionalmente filtradas)

Cierra en bloque. Devuelve el resultado de CADA cierre por separado: si una falla, las demas siguen adelante y el fallo queda registrado.

Cuerpo de la peticion — `application/json`

Campo	Tipo	Obligatorio	Descripcion
symbol	string?	no	Cerrar solo las posiciones de este simbolo. null = todas.
magic	integer (int64)?	no	Cerrar solo las posiciones con este magic. null = cualquiera.
comment	string?	no	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST `/trading/close-partial/{ticket}` requiere token

Cerrar parcialmente una posicion

Igual que `/trading/close/{ticket}` pero `volume` es OBLIGATORIO.

Existe como endpoint propio porque es la operacion que mas se usa desde un cliente y asi el contrato es explicito.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	Ticket de la posicion abierta.

Cuerpo de la peticion — `application/json`

Campo	Tipo	Obligatorio	Descripcion
volume	number (double)?	no	Volumen a cerrar. null o ausente = cerrar la posicion ENTERA.
deviation	integer (int64)?	no	Slippage maximo en puntos.
comment	string?	no	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST `/trading/close/{ticket}` requiere token

Cerrar posicion (total o parcial)

Sin `volume` cierra la posicion entera. Con `volume` menor que el de la posicion hace un CIERRE PARCIAL y el resto sigue abierto.

En cuentas **hedging** el cierre parcial deja la posicion original con el volumen restante y su mismo ticket. En cuentas **netting** el resultado es una unica posicion neta.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	Ticket de la posicion abierta.

Cuerpo de la peticion — `application/json`

Campo	Tipo	Obligatorio	Descripcion
volume	number (double)?	no	Volumen a cerrar. null o ausente = cerrar la posicion ENTERA.
deviation	integer (int64)?	no	Slippage maximo en puntos.
comment	string?	no	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/modify-sl/{ticket}

requiere token

Mover solo el Stop Loss

El TP no se toca. `sl: 0` elimina el Stop Loss.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
sl	number (double)	si	Nuevo Stop Loss como precio absoluto. 0 = quitar el SL.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/modify-tp/{ticket}

requiere token

Mover solo el Take Profit

El SL no se toca. `tp: 0` elimina el Take Profit.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
tp	number (double)	si	Nuevo Take Profit como precio absoluto. 0 = quitar el TP.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/modify/{ticket}

requiere token

Mover SL y/o TP de una posicion

Omite un campo (`null`) para dejarlo como esta. Manda `0` para QUITARLO.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
sl	number (double)?	no	Nuevo Stop Loss (precio absoluto). Enviar 0 para QUITAR el SL. Omitir (null) para dejarlo como esta.
tp	number (double)?	no	Nuevo Take Profit (precio absoluto). Enviar 0 para QUITAR el TP. Omitir (null) para dejarlo como esta.

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/open requiere token

Abrir posicion a mercado

Ejecuta BUY o SELL a mercado.

El SL/TP se pueden dar de dos formas: como PRECIO (`sl`, `tp`) o como DISTANCIA en puntos (`sl\_points`, `tp\_points`). Si mandas ambas, gana el precio absoluto. Un punto es `symbol\_info.point` (para EURUSD de 5 dígitos, 1 punto = 0,00001; 10 puntos = 1 pip).

Cuerpo de la petición — `application/json`

Campo	Tipo	Obligatorio	Descripción
symbol	string	si	Nombre exacto del símbolo en el broker.
side	"BUY" "SELL"	si	BUY o SELL. Ejecución a mercado.
volume	number (double)	si	Lotes. Se ajusta al volume_step del símbolo, siempre hacia abajo.
sl	number (double)?	no	Stop Loss como PRECIO absoluto. null = sin SL.
tp	number (double)?	no	Take Profit como PRECIO absoluto. null = sin TP.
sl_points	integer (int64)?	no	Alternativa a `sl`: distancia en puntos desde el precio de entrada. Se ignora si `sl` viene informado.
tp_points	integer (int64)?	no	Alternativa a `tp`: distancia en puntos desde el precio de entrada.
deviation	integer (int64)?	no	Slippage máximo aceptado, en puntos. Por defecto el del gateway (20).
magic	integer (int64)?	no	Magic number de la orden. Por defecto el del gateway.
comment	string?	no	Comentario libre. MT5 lo trunca a 31 caracteres.

Sin ejemplo capturado en la última pasada del selftest (por ejemplo, porque dependía de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/pending requiere token

Colocar orden pendiente (limit / stop)

Cuerpo de la petición — `application/json`

Campo	Tipo	Obligatorio	Descripción
symbol	string	si	
type	"BUY_LIMIT" "SELL_LIMIT" "BUY_STOP" "SELL_STOP"	si	Tipo de orden pendiente.
volume	number (double)	si	
price	number (double)	si	Precio de activación de la orden.
sl	number (double)?	no	
tp	number (double)?	no	
stoplimit	number (double)?	no	Solo para BUY_STOP_LIMIT / SELL_STOP_LIMIT.
expiration	integer (int64)?	no	Epoch en segundos (hora de servidor). null = GTC (sin caducidad).
magic	integer (int64)?	no	
comment	string?	no	

Sin ejemplo capturado en la última pasada del selftest (por ejemplo, porque dependía de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

DELETE /trading/pending/{ticket} requiere token

Cancelar una orden pendiente

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripción
ticket	path	integer (int64)	si	—	

Sin ejemplo capturado en la última pasada del selftest (por ejemplo, porque dependía de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

POST /trading/pending/{ticket}/modify

requiere token

Modificar una orden pendiente

Cambia precio de activacion, SL, TP o caducidad. Los campos `null` se dejan como estan.

Parametros (URL / query)

Nombre	En	Tipo	Obligatorio	Por defecto	Descripcion
ticket	path	integer (int64)	si	—	

Cuerpo de la peticion — application/json

Campo	Tipo	Obligatorio	Descripcion
price	number (double)?	no	Nuevo precio de activacion. null = dejarlo igual.
sl	number (double)?	no	
tp	number (double)?	no	
stoplimit	number (double)?	no	
expiration	integer (int64)?	no	

Sin ejemplo capturado en la ultima pasada del selftest (por ejemplo, porque dependia de un ticket que ya no existe). El contrato es el descrito en las tablas de arriba.

7. Tablas de codigos

El gateway ya envia el nombre junto al numero (por ejemplo `type` y `type_name`), asi que normalmente no necesitas estas tablas. Van aqui por si prefieres trabajar con los valores numericos en .NET.

Tipo de orden (`type` en /orders y en el resultado)

Valor	Significado
0	BUY
1	SELL
2	BUY_LIMIT
3	SELL_LIMIT
4	BUY_STOP
5	SELL_STOP
6	BUY_STOP_LIMIT
7	SELL_STOP_LIMIT
8	CLOSE_BY

Tipo de posicion (`type` en /positions)

Valor	Significado
0	BUY
1	SELL

Estado de orden (`state`)

Valor	Significado
0	STARTED
1	PLACED
2	CANCELED
3	PARTIAL
4	FILLED
5	REJECTED
6	EXPIRED
7	REQUEST_ADD
8	REQUEST_MODIFY
9	REQUEST_CANCEL

Tipo de deal (type en /history/deals)

Valor	Significado
0	BUY
1	SELL
2	BALANCE
3	CREDIT
4	CHARGE
5	CORRECTION
6	BONUS
7	COMMISSION
12	INTEREST
15	DIVIDEND
17	TAX

Direccion del deal (entry)

Valor	Significado
0	IN (abre)
1	OUT (cierra)
2	INOUT (invierte)
3	OUT_BY (cierre por opuesta)

Motivo del deal (reason)

Valor	Significado
0	CLIENT
1	MOBILE
2	WEB
3	EXPERT
4	SL alcanzado
5	TP alcanzado
6	SO (stop out)
7	ROLLOVER
8	VMARGIN
9	SPLIT

Tipo de cuenta (trade_mode)

Valor	Significado
0	DEMO
1	CONTEST
2	REAL

Modo de margen (margin_mode)

Valor	Significado
0	RETAIL_NETTING
1	EXCHANGE
2	RETAIL_HEDGING

Retcodes de trading mas frecuentes

Codigo	Nombre	Que significa
10004	REQUOTE	Requote
10006	REJECT	Peticion rechazada
10009	DONE	Ejecutada correctamente
10010	DONE_PARTIAL	Ejecutada parcialmente

10013	INVALID	Peticion invalida
10014	INVALID_VOLUME	Volumen invalido
10015	INVALID_PRICE	Precio invalido
10016	INVALID_STOPS	SL/TP invalidos (demasiado cerca del precio)
10017	TRADE_DISABLED	Trading deshabilitado
10018	MARKET_CLOSED	Mercado cerrado
10019	NO_MONEY	Fondos insuficientes
10020	PRICE_CHANGED	El precio cambio
10027	CLIENT_DISABLES_AT	AutoTrading desactivado en el terminal
10030	INVALID_FILL	Modo de llenado no soportado
10031	CONNECTION	Sin conexion
10036	POSITION_CLOSED	La posicion ya estaba cerrada
10038	INVALID_CLOSE_VOLUME	Volumen de cierre superior al de la posicion

La lista completa de retcodes esta en `app/converters.py` (`RETCODE_NAMES`).